Постановка задачи

Разработать программу на языке Python, решающую задачу поиска глобального экстремума для заданной одномерной Липшицевой функции на заданном отрезке методом ломаных.

На вход программа принимает строку одномерной функции, координаты концов отрезка (вещественные числа), точность вычисления eps. На выходе она выполняет визуализацию графика исходной функции, вспомогательных функций (по возможности итоговой ломаной, на которой достигнута требуемая точность), приближенное значение аргумента и минимального значения функции, число итераций, потраченной время.

В качестве тестовой задачи используется функция Растригина:

$$W(x) = x^2 - 10\cos(2\pi x) + 10, \quad x \in [-1.5; 1.5], \quad \varepsilon = 0.01.$$

Метод ломаных (Пиявского)

Функция называется Липшицевой на отрезке $[a, b]$, если существует константа $L > 0$ такая, что для любых $u, v \in [a, b]$ выполняется $|W(u) - W(v)| \leq L \cdot |u - v|$. Для гладких функций $L \geq \max |W'(x)|$.

В методе ломаных поиск оптимума ведётся на вспомогательной кусочно-линейной функции — миноранте $p(x) \leq W(x)$, составленной из лучей с наклонами $\pm L$, опущенных из верхних вершин $(u_i, W(u_i))$. Координаты нижних вершин (точек пересечения лучей u_{cross}) между соседними точками u_q и u_v рассчитываются по формулам:

$$u_{cross} = \frac{W(u_q) - W(u_v)}{2L} + \frac{u_q + u_v}{2},$$

$$p(u_{cross}) = \frac{W(u_q) + W(u_v)}{2} - \frac{L(u_v - u_q)}{2}.$$

Итерации завершаются при выполнении условия: $\delta = |W(u_{new}) - p(u_{cross})| < \varepsilon$.

Расчёт константы Липшица

Функция calculate_lipschitz строит сетку из 2000 точек и вычисляет максимальное численное отношение приращений dy / dx. Результат умножается

на коэффициент надёжности 1.1 (10% запаса), чтобы ломаная гарантированно не пересекала функцию сверху:

```
def calculate_lipschitz(f, a, b, samples=2000):
    grid = np.linspace(a, b, samples)
    y_vals = np.array([f(pt) for pt in grid])
    dy = np.abs(np.diff(y_vals))
    dx = np.abs(np.diff(grid))
    l_calc = float(np.max(dy / dx) * 1.1)
    return max(l_calc, 1)
```

Обработка граничных условий

В начале функции piyavsky_method строка формулы преобразуется в вызываемый объект через функцию eval с математическим словарём функций. Проверяется вырожденность отрезка ($a = b$), при необходимости меняются местами границы ($a > b$), а при поиске максимума знак функции инвертируется ($W \to -W$):

```
    raw_f = lambda x: eval(func_str, {"x": x, "sin": np.sin, "cos": np.cos,
   "tan": np.tan, "exp": np.exp, "log": np.log, "sqrt": np.sqrt, "abs": np.
   abs, "pi": np.pi, "e": np.e})

    try:
        raw_f(a)
    except Exception:
        print("\nНеправильно записана формула функции")
        return

    if a == b:
        print(f"\nГраницы совпадают. Отрезок равен точке.")
        return a, 0, 0, 0

    if a > b:
        a, b = b, a

    sign = -1 if mode == 'max' else 1
    f = lambda x: sign * raw_f(x)
    L = calculate_lipschitz(f, a, b)
```

Алгоритм поиска минимума ломаной

В цикле while верхние вершины сортируются по оси x, после чего рассчитываются координаты всех нижних вершин u_cross. Выбирается самая глубокая точка миноранты best_u, в которой вычисляется реальное значение функции, и проверяется критерий останова:

```
start_time = time.perf_counter()

upper_u = [float(a), float(b)]
upper_w = [float(f(a)), float(f(b))]

iterations = 0

while True:
    iterations += 1

    if iterations >= max_iter:
        print(f"\nДостигнут лимит в {max_iter} итераций")
        break

    order = np.argsort(upper_u)
    upper_u = [upper_u[i] for i in order]
    upper_w = [upper_w[i] for i in order]

    best_p = np.inf
    best_u = None

    for i in range(len(upper_u) - 1):
        uq, uv = upper_u[i], upper_u[i + 1]
        wq, wv = upper_w[i], upper_w[i + 1]

        u_cross = (wq - wv) / (2 * L) + (uq + uv) / 2
        p_val = (wq + wv) / 2 - L * (uv - uq) / 2

        if p_val < best_p:
            best_p = p_val
            best_u = u_cross

    new_u = best_u
    new_w = float(f(new_u))

    delta = abs(new_w - best_p)
```

```
        if delta < eps:
            upper_u.append(new_u)
            upper_w.append(new_w)
            break

        upper_u.append(new_u)
        upper_w.append(new_w)

    exec_time = time.perf_counter() - start_time

    best_idx = int(np.argmin(upper_w))
    x_opt = upper_u[best_idx]
    w_opt = sign * upper_w[best_idx]
```

Вывод данных и построение графика

После завершения расчётов числовые показатели выводятся в консоль, а затем вызывается функция plot_results:

```
    print(f"\nРежим оптимизации: {mode}")
    print(f"Отрезок: [{a}, {b}]")
    print(f"Константа Липшица L: {round(L, 4)}")
    print(f"Точка экстремума x*: {round(x_opt, 5)}")
    print(f"Значение функции W(x*): {round(w_opt, 5)}")
    print(f"Число проведенных итераций: {iterations}")
    print(f"Затраченное время: {exec_time:.5f} сек")

    plot_results(raw_f, a, b, upper_u, upper_w, L, x_opt, w_opt, func_str,
   sign)

    return x_opt, w_opt, iterations, exec_time
```

Модуль отрисовки графика

Функция соединяет вершины в ломаную линию миноранты и наносит найденную точку экстремума:

```
def plot_results(f, a, b, upper_u, upper_w, L, x_opt, w_opt, func_str, sign)
   :
    x_grid = np.linspace(a, b, 1000)
    y_grid = [f(x) for x in x_grid]

    plt.figure(figsize=(10, 6), dpi=100)
```

```
 plt.plot(x_grid, y_grid, label=f"W(x) = {func_str}", color="blue",
linewidth=2)

 order = np.argsort(upper_u)
 xs = [upper_u[i] for i in order]
 ws = [upper_w[i] for i in order]

 broken_x = []
 broken_y = []

 for i in range(len(xs) - 1):
     uq, uv = xs[i], xs[i + 1]
     wq, wv = ws[i], ws[i + 1]

     u_cross = (wq - wv) / (2 * L) + (uq + uv) / 2
     p_val = (wq + wv) / 2 - L * (uv - uq) / 2

     broken_x.extend([uq, u_cross])
     broken_y.extend([sign * wq, sign * p_val])

 broken_x.append(xs[-1])
 broken_y.append(sign * ws[-1])

 plt.plot(broken_x, broken_y, '--', color="orange", label="Ломаная линия"
, linewidth=1.5)
 plt.scatter([x for x in upper_u], [f(x) for x in upper_u], color="black"
, s=10, label="Вершины", zorder=3)
 plt.scatter([x_opt], [w_opt], color="red", s=30, zorder=5, edgecolor="
black", linewidth=1, label=f"Экстремум ({round(x_opt, 3)}; {round(w_opt,
 3)})")

 plt.title("Метод ломаных (Пиявского)", fontsize=12)
 plt.xlabel("x")
 plt.ylabel("W(x)")
 plt.grid(True, linestyle=":", alpha=0.6)
 plt.legend()
 plt.tight_layout()
 plt.show()
```

Безопасный ввод числовых данных

Функция get_float_input предотвращает аварийное завершение програм-

мы при вводе запятой вместо точки или буквенных символов:

```
def get_float_input(prompt):
    while True:
        user_input = input(prompt).replace(',', '.')
        try:
            return float(user_input)
        except ValueError:
            print("Пожалуйста, введите число")
```

Результаты экспериментов

Программа была запущена для тестовой функции Растригина. На рисунках 1 и 2 представлены численные результаты работы программы в консоли и графическая иллюстрация найденного решения.

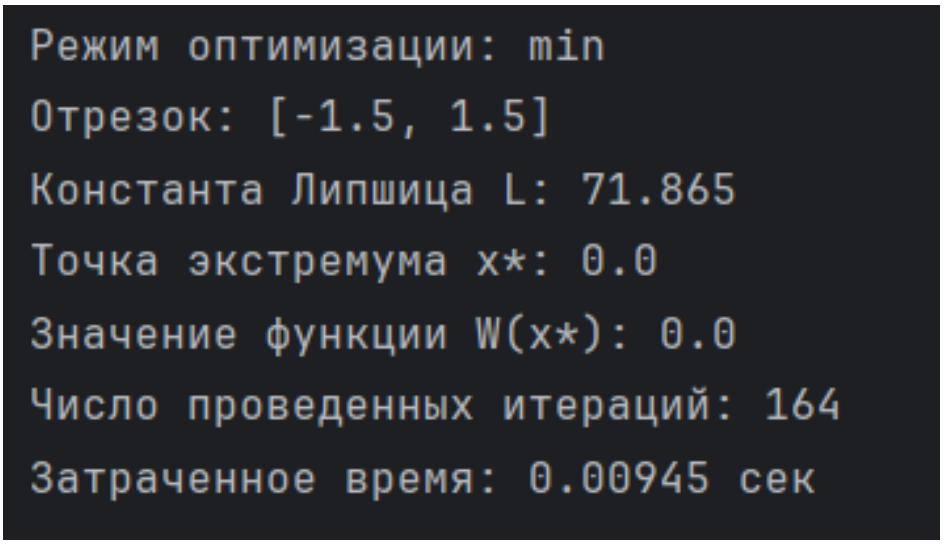


Рисунок 1 — Результаты работы программы в консоли

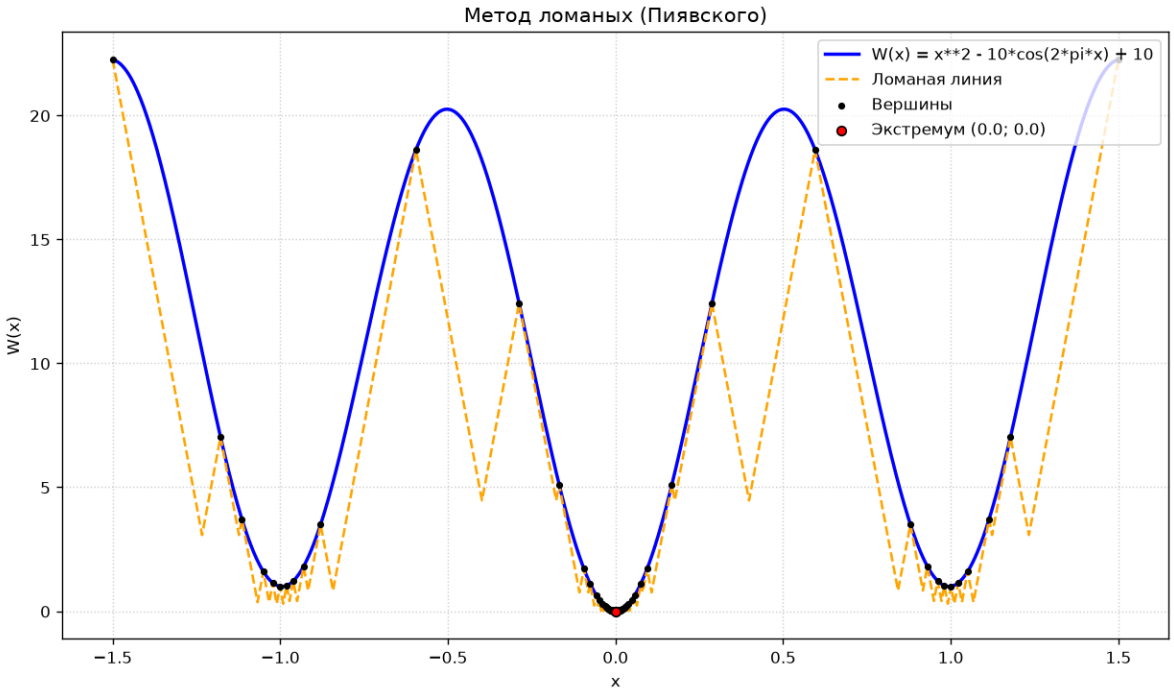


Рисунок 2 — График функции Растригина и построенная миноранта Пиявского

Вывод

В ходе работы был изучен и реализован метод ломаных (Пиявского) для поиска глобального минимума функции на отрезке. Тестирование на функции Растригина подтвердило, что метод успешно находит истинный глобальный минимум и не застревает в локальных экстремумах. Результаты расчётов и построенный график ломаной линии наглядно подтвердили сходимость алгоритма и достижение заданной точности.